

01. Real Production Scenario: Incremental Data Load

PRODUCTION SCENARIO

Every night, only new customer records should be copied from SQL Server to Azure Data Lake instead of loading the entire table.

INTERVIEW QUESTION

What would be your first approach to implement incremental loading in Azure Data Factory?

MY ANSWER

My first approach would be to identify a reliable watermark column in the source, such as `CreatedDate`, `LastModifiedDate`, or an increasing ID. I would store the last successfully processed watermark in a control table rather than hard-coding it in the pipeline. At runtime, ADF would read the previous watermark, query only records greater than that value, write them to the raw ADLS layer, and update the watermark only after the load and validation succeed.

I would also keep pipeline-run and batch metadata so that the same window can be replayed if required. For updates, I would prefer a last-modified timestamp or CDC pattern rather than an ID that only represents inserts. The important production rule is: never advance the watermark after a partial or failed load. Microsoft documents this watermark pattern for incremental copy and supports an increasing ID or last-modified column as the slicing value.

FOLLOW-UP QUESTIONS

1. What would you do if the source has both inserts and updates?
2. Where would you store the watermark and why?
3. How would you make the load idempotent?
4. What happens if the pipeline fails after copying but before updating the watermark?
5. How would you backfill a missed date range?

02. Real Production Scenario: Late Arriving Files

PRODUCTION SCENARIO

Your ADF pipeline runs at 1 AM, but sometimes source files arrive at 2 AM, causing pipeline failures.

INTERVIEW QUESTION

How would you design the pipeline to handle late-arriving files?

MY ANSWER

I would not make the pipeline depend on a single exact arrival time. I would separate file arrival from processing. One option is an event-based trigger when the file lands; another is a scheduled polling pattern that checks the expected folder for a defined window. I would use Get Metadata or file-listing logic to identify whether the expected file exists, then use an Until loop or controlled retry strategy with a maximum waiting period.

If the file is still missing after the SLA window, I would mark the batch as delayed and alert the support team instead of silently failing. Once the file arrives, the pipeline should process it using a deterministic file name or business date, record the file in an audit/control table, and prevent duplicate processing. For critical feeds, I would also separate 'file received' status from 'file successfully processed' status so operations can see exactly where the delay occurred.

FOLLOW-UP QUESTIONS

1. Would you use a schedule trigger or event trigger here?
2. How would you avoid processing the same file twice?
3. What would happen if the file arrives after the maximum wait time?
4. How would you monitor missing files?
5. How would you handle multiple files for the same business date?

03. Real Production Scenario: Dynamic Pipeline

PRODUCTION SCENARIO

One ADF pipeline should load data from 50 different source tables without creating 50 separate pipelines.

INTERVIEW QUESTION

How would you make the pipeline dynamic?

MY ANSWER

I would build a metadata-driven pipeline. Instead of hard-coding table names, source queries, destination paths, load types, watermark columns, and target objects, I would store those values in a configuration table. The pipeline would use a Lookup activity to read active configurations and a ForEach activity to process each table. Parameters would be passed into reusable datasets and linked services, so the same copy logic can handle many sources.

I would include columns such as SourceSchema, SourceTable, TargetPath, LoadType, WatermarkColumn, PrimaryKey, IsActive, and LastWatermark. This makes onboarding a new table mostly a metadata change rather than a new pipeline. I would also capture each table's execution status, row counts, start/end time, and error message in an audit table. For production, I would control ForEach parallelism carefully so that 50 tables do not overload the source database.

FOLLOW-UP QUESTIONS

1. What columns would you keep in the metadata table?
2. How would you handle different source schemas?
3. How would you control parallelism?
4. How would you handle one table failing while others succeed?
5. How would you onboard a new table without changing the pipeline?

04. Real Production Scenario: Corrupted File Handling

PRODUCTION SCENARIO

One CSV file is corrupted, but the remaining 99 files should still be processed successfully.

INTERVIEW QUESTION

How would you implement error handling in ADF?

MY ANSWER

I would design the file-processing pattern so one bad file does not stop the entire batch. If the files can be processed independently, I would enumerate them and process each file separately inside a ForEach activity. The per-file branch would have success and failure paths. A successful file would be written to the curated destination and then moved to Archive. A failed or malformed file would be copied or moved to a Reject area, with the file name, error message, pipeline run ID, and timestamp recorded in an audit table.

ADF Copy Activity also has fault-tolerance options for certain incompatible data scenarios, and session logging can capture skipped data. I would use that where appropriate, but I would not treat fault tolerance as a replacement for validation. For malformed CSV structure, encoding problems, or business-rule failures, I would explicitly quarantine the file and alert operations.

FOLLOW-UP QUESTIONS

1. Would you fail the whole pipeline or continue processing?
2. What information would you log for a rejected file?
3. How would you retry only the failed file?
4. How would you distinguish technical errors from data-quality errors?
5. When would Copy Activity fault tolerance be appropriate?

05. Real Production Scenario: Large File Processing

PRODUCTION SCENARIO

A daily transaction file has grown to 500 GB, and the pipeline is missing its SLA.

INTERVIEW QUESTION

How would you optimize the ingestion process?

MY ANSWER

I would first measure where the time is being spent instead of immediately increasing compute. I would check source read speed, network throughput, Integration Runtime performance, sink write speed, file format, partitioning, and any transformations happening during ingestion. If the source is a large CSV, I would consider converting the raw data to a columnar format such as Parquet as early as practical, while keeping the original raw copy if required for replay.

I would also avoid unnecessary transformations in the ingestion step and separate raw landing from heavy processing. For independent files or partitions, I would use controlled parallelism rather than serial processing. I would check whether the source supports partitioned extraction and whether staging can improve throughput for the chosen connector. Finally, I would benchmark changes and compare throughput, duration, cost, and SLA rather than assuming that a larger cluster is automatically better.

FOLLOW-UP QUESTIONS

1. How would you identify the bottleneck?
2. When would you use parallel copy?
3. Would you transform the 500 GB file during ingestion?
4. How would file format affect performance?
5. How would you prove the optimization improved the SLA?

06. Real Production Scenario: AWS S3 to Azure Data Lake

PRODUCTION SCENARIO

Raw files are stored in AWS S3, but the analytics team uses Azure Synapse.

INTERVIEW QUESTION

How would you build a secure pipeline from AWS S3 to Azure Data Lake?

MY ANSWER

I would use ADF Copy Activity with Amazon S3 as the source and ADLS Gen2 as the destination, then let Synapse consume the curated data from Azure. I would keep the S3 source read-only for this integration and separate raw, validated, and curated zones in ADLS. For credentials, I would avoid hard-coding access keys in the pipeline and store secrets in Azure Key Vault, with ADF using managed identity to retrieve them.

I would also restrict network access, apply least-privilege permissions, enable logging, and record source object names and batch IDs for traceability. If the S3 bucket is private, I would verify the required connectivity and authentication model supported by the ADF S3 connector and the organization's security architecture. After landing, I would validate file counts and sizes before marking the batch successful.

FOLLOW-UP QUESTIONS

1. How would you secure the S3 credentials?
2. Would you copy directly into Synapse? Why or why not?
3. How would you validate that all S3 objects arrived?
4. How would you handle a partially copied batch?
5. What would you log for cross-cloud lineage?

07. Real Production Scenario: Duplicate Records

PRODUCTION SCENARIO

Duplicate customer records appear after every pipeline execution.

INTERVIEW QUESTION

How would you identify and remove duplicate data?

MY ANSWER

I would first determine whether the duplicates are created by the source, by the extraction query, or by the target load pattern. I would identify the business key, such as `CustomerID`, and inspect duplicates with a `GROUP BY` and `COUNT` check. If the source itself contains multiple versions, I would use a deterministic rule such as the latest `LastModified` timestamp and a window function to retain one record per business key.

At the pipeline level, I would make the load idempotent: the same batch should not create a second copy when it is retried. Depending on the target, I might load to a staging table and then `MERGE` or use a Delta Lake upsert pattern. I would also maintain a batch or source-file identifier so I can trace exactly which execution introduced a duplicate. The goal is not simply to delete duplicates after the fact; it is to identify the root cause and make retries safe.

FOLLOW-UP QUESTIONS

1. What would be your business key?
2. How would `ROW_NUMBER` help?
3. How would you prevent duplicates during retries?
4. Would you deduplicate in ADF, Databricks, or SQL?
5. How would you prove the duplicate issue is fixed?

08. Real Production Scenario: Slowly Changing Dimension (SCD Type 2)

PRODUCTION SCENARIO

Customer addresses change frequently, but historical values must be preserved.

INTERVIEW QUESTION

How would you implement SCD Type 2 in a production pipeline?

MY ANSWER

I would maintain a dimension table with a surrogate key, the business key such as CustomerID, effective-from and effective-to timestamps, and an IsCurrent flag. For each incoming customer record, I would compare the relevant attributes with the current dimension record. If nothing changed, I would leave the current row unchanged. If the address changed, I would expire the existing row by setting its end timestamp and IsCurrent to false, then insert a new row with the new address, a new effective timestamp, and IsCurrent true.

I would perform this in a controlled transaction or staged process where the target technology supports it. I would also handle late-arriving changes carefully because the effective business date may differ from the ingestion time. Before release, I would test inserts, updates, repeated identical records, and late-arriving changes. The important point is that the old address remains queryable for historical reporting.

FOLLOW-UP QUESTIONS

1. What columns are required for SCD Type 2?
2. How would you detect a changed address?
3. What happens when the same record arrives twice?
4. How would you handle a late-arriving historical change?
5. How would you query the address that was valid on a past date?

09. Real Production Scenario: Failed Pipeline Recovery

PRODUCTION SCENARIO

An ADF pipeline fails after processing 80% of the data.

INTERVIEW QUESTION

How would you restart the pipeline without reprocessing completed records?

MY ANSWER

I would design recovery into the pipeline before the first production run. For file-based ingestion, I would maintain a control table containing file name, batch ID, status, row count, start time, end time, and error information. Completed files would be marked SUCCESS, so a rerun processes only PENDING or FAILED files. For database loads, I would use a watermark or batch boundary and only advance the checkpoint after successful completion.

The target write should also be idempotent, using a natural key, batch key, MERGE, or overwrite-by-partition strategy where appropriate. I would avoid a single giant transaction if the source is large and independent partitions can be committed safely. During recovery, I would first identify exactly what was committed before the failure, then restart from the last confirmed checkpoint. This prevents both data loss and uncontrolled duplication.

FOLLOW-UP QUESTIONS

1. What would you store in the control table?
2. How do you know which records were already committed?
3. How would you make the sink idempotent?
4. When would you use a checkpoint versus a watermark?
5. How would you recover from a failure during the final commit?

10. Real Production Scenario: Secure Credentials

PRODUCTION SCENARIO

Your pipeline connects to multiple databases, but credentials cannot be stored directly inside ADF.

INTERVIEW QUESTION

How would you securely manage secrets?

MY ANSWER

I would use Azure Key Vault for secrets and have ADF access the vault using its managed identity. Linked services can reference Key Vault secrets instead of embedding passwords or connection secrets directly in pipeline JSON. I would use separate vaults or controlled secret scopes for different environments where appropriate, apply least-privilege access, and rotate secrets without changing pipeline logic.

I would also avoid exposing secrets through parameters, debug output, logs, or source control. For connections that support managed identity or service principals, I would prefer identity-based authentication over static credentials. The production design should make the pipeline code reusable while keeping environment-specific secrets outside the code. Microsoft specifically recommends storing secrets in Azure Key Vault and using managed identity for secure ADF access.

FOLLOW-UP QUESTIONS

1. Why is Key Vault better than storing a password in a parameter?
2. How would ADF authenticate to Key Vault?
3. How would you rotate a database password?
4. How would you separate Dev and Prod secrets?
5. How would you prevent secrets from appearing in logs?

11. Real Production Scenario: Multi-Environment Deployment

PRODUCTION SCENARIO

The same ADF solution must run in Dev, QA, and Production using different connection strings.

INTERVIEW QUESTION

How would you manage environment-specific configurations?

MY ANSWER

I would keep the pipeline logic the same and parameterize values that change by environment. Typical examples are storage account names, database names, Key Vault names, endpoints, and feature flags. I would use ADF parameters and global parameters where appropriate, and parameterize linked services when the connection properties need to vary. CI/CD would deploy the same solution to Dev, QA, and Prod with environment-specific parameter values.

Secrets would remain in the corresponding Key Vault rather than being committed to source control. Microsoft documents global parameters as a way to override environment-specific values during CI/CD, and linked services can also be parameterized. I would additionally use separate access permissions for production resources and require validation gates before production deployment. This gives us one maintainable codebase with controlled environment configuration.

FOLLOW-UP QUESTIONS

1. Which values should be parameterized?
2. Where would you store passwords?
3. How would the same pipeline connect to different databases?
4. How would you protect production configuration?
5. How would you roll back a bad deployment?

12. Real Production Scenario: Data Validation

PRODUCTION SCENARIO

Before loading data into the warehouse, row count, null values, and schema validation must be completed.

INTERVIEW QUESTION

How would you automate these validations in ADF?

MY ANSWER

I would create a validation stage between ingestion and the warehouse load. First, I would calculate expected and actual row counts using SQL queries, file metadata, or source control totals. Next, I would run business-quality checks such as CustomerID not null, valid dates, acceptable numeric ranges, and duplicate-key checks. For schema validation, I would compare the incoming structure with the expected metadata and explicitly decide whether new columns are allowed.

The validation result would be written to an audit table with BatchID, check name, expected value, actual value, status, and error message. ADF can branch using If Condition or validation activities so the warehouse load proceeds only when critical checks pass. Failed batches should be quarantined and alerted rather than partially published. For reusable frameworks, I would keep the validation rules in metadata so the same pattern can be applied to many datasets.

FOLLOW-UP QUESTIONS

1. Which checks are technical and which are business checks?
2. Where would you store validation results?
3. What should happen when one critical check fails?
4. How would you validate schema changes?
5. How would you make validation reusable across 50 tables?

13. Real Production Scenario: AWS Glue vs Azure Data Factory

PRODUCTION SCENARIO

Your company uses both AWS and Azure. Management asks whether the ETL pipeline should be built using AWS Glue or Azure Data Factory.

INTERVIEW QUESTION

What factors would you consider before making the decision?

MY ANSWER

I would not choose purely from the product name. I would first map the workload to the cloud where the main data, security boundaries, and downstream analytics services already exist. I would compare connector coverage, transformation requirements, orchestration, integration runtime or compute model, networking, identity, monitoring, CI/CD, operational skills, cost, and expected data volume. If the workload is heavily AWS-native and transformation is Spark-centric, Glue may fit naturally.

If the solution is centered on Azure services such as ADLS, Synapse, Key Vault, and Azure monitoring, ADF may reduce integration complexity. For a cross-cloud platform, I would also consider where data movement occurs and whether repeated cross-cloud transfers add cost or latency. I would run a small proof of concept using representative data and measure throughput, operational effort, security requirements, and total cost. The final decision should be documented against business and technical requirements rather than personal preference.

FOLLOW-UP QUESTIONS

1. Which workloads would favor Glue?
2. Which workloads would favor ADF?
3. How would you compare total cost?
4. How would cross-cloud data transfer affect the decision?
5. What would you include in a proof of concept?

14. Real Production Scenario: Event-Based Pipeline Trigger

PRODUCTION SCENARIO

The pipeline should automatically start whenever a new file is uploaded to Azure Blob Storage.

INTERVIEW QUESTION

How would you implement an event-driven pipeline instead of a scheduled pipeline?

MY ANSWER

I would use an ADF storage event trigger so the pipeline starts when the configured storage event occurs, rather than polling on a fixed schedule. The trigger can pass file-related values into pipeline parameters, allowing the pipeline to process the specific object instead of scanning the entire folder. I would still make the processing idempotent because event-driven systems can require defensive handling of duplicate or unexpected events.

I would validate the file before processing, write an audit record, and move or mark the file after successful completion. For production, I would also monitor trigger failures, processing latency, and files that remain unprocessed. If the upstream system uploads temporary or incomplete objects, I would use an agreed file-arrival contract, such as writing to a temporary location and publishing the final file only when complete.

FOLLOW-UP QUESTIONS

1. What information would you pass from the trigger to the pipeline?
2. How would you avoid duplicate processing?
3. What if the file is still being uploaded?
4. When is event-based triggering better than scheduling?
5. How would you monitor unprocessed files?

15. Real Production Scenario: End-to-End Data Pipeline Design

PRODUCTION SCENARIO

An e-commerce platform generates millions of orders every day, and dashboards must refresh every hour.

INTERVIEW QUESTION

How would you design a scalable end-to-end data pipeline using Azure and AWS services?

MY ANSWER

I would first define the system of record and decide where the primary analytics platform should live. A possible hybrid design is to ingest operational data and AWS-hosted files into a controlled raw layer in ADLS Gen2, orchestrated by ADF. Raw data would be immutable and replayable. Databricks could perform scalable cleansing, deduplication, enrichment, and incremental transformations into Delta tables using a bronze, silver, and gold pattern.

Curated data could then be served through Synapse or another SQL-serving layer for BI. I would partition processing by business date or another appropriate key, use incremental processing rather than full reloads, and maintain audit/checkpoint metadata. Security would use managed identities, Key Vault, least-privilege access, and private connectivity where required. For the hourly SLA, I would define freshness and volume metrics, monitor each stage, and design retries so transient failures recover without duplicating data.

FOLLOW-UP QUESTIONS

1. Where would you keep the raw data?
2. Why separate bronze, silver, and gold layers?
3. How would you meet the one-hour freshness SLA?
4. How would you handle cross-cloud failures?
5. What metrics would you monitor end to end?

16. Real Production Scenario: API Data Ingestion

PRODUCTION SCENARIO

Your company receives customer data from a third-party REST API every hour. Sometimes the API returns partial data or rate-limit errors.

INTERVIEW QUESTION

How would you build a reliable ingestion pipeline?

MY ANSWER

I would design the API ingestion as a checkpointed process rather than a single request. The pipeline would store the API endpoint configuration, authentication reference, pagination state, and last successful business watermark. For each run, it would request pages, handle continuation tokens or page numbers, and write raw responses to a landing area before transformation. If the API returns HTTP 429 or a transient server error, I would use bounded retries with increasing delays and respect the provider's rate-limit guidance.

For partial responses, I would validate the expected page or record counts where the API provides them and avoid advancing the checkpoint until the complete business window is successfully ingested. I would also persist request metadata, response status, and batch ID for troubleshooting. If a run fails halfway through, the next run should be able to replay the same window safely without creating duplicates.

FOLLOW-UP QUESTIONS

1. How would you handle HTTP 429?
2. How would you implement pagination?
3. What would you use as the API watermark?
4. How would you avoid duplicate data after a retry?
5. How would you detect partial API responses?

17. Real Production Scenario: Retry Mechanism

PRODUCTION SCENARIO

A database connection occasionally fails due to network issues, but succeeds after a few minutes.

INTERVIEW QUESTION

How would you implement automatic retry without manual intervention?

MY ANSWER

I would configure activity retry and retry interval for transient operations, but I would first confirm that retrying the operation is safe. A retry is appropriate for temporary network or service availability errors, but not for deterministic SQL errors such as invalid credentials or an invalid query. I would make the pipeline idempotent so a retry cannot create duplicate business records. For example, I would use a batch identifier, merge strategy, or write to staging before committing to the target.

I would also set a reasonable retry count and interval based on the service's expected recovery time. Monitoring should distinguish between an activity that succeeded after retry and one that permanently failed. This gives operations visibility without requiring someone to restart every transient failure manually. For critical pipelines, I would pair retries with alerts after the final retry is exhausted.

FOLLOW-UP QUESTIONS

1. Which failures should not be retried?
2. How many retries would you configure?
3. How do you prevent duplicate data during retries?
4. What should happen after the final retry fails?
5. How would you monitor repeated transient failures?

18. Real Production Scenario: Parallel Processing

PRODUCTION SCENARIO

You need to process 200 source files every hour, but sequential execution is too slow.

INTERVIEW QUESTION

How would you improve pipeline performance?

MY ANSWER

I would parallelize independent files rather than processing all 200 sequentially. In ADF, I would use a Lookup to obtain the file list and a ForEach activity with controlled batch count. I would start with a safe level of concurrency and increase it after measuring source throughput, Integration Runtime capacity, sink throughput, and downstream pressure.

I would avoid unlimited parallelism because the source database, storage account, network, or sink can become the bottleneck.

I would also consider whether the files can be combined or converted to a more efficient format to reduce overhead from many tiny objects. Each file should have its own audit status so one failure does not force all successful files to be rerun. Finally, I would compare total runtime before and after the change and monitor throttling or resource saturation.

FOLLOW-UP QUESTIONS

1. What controls the number of concurrent files?
2. Why can too much parallelism reduce performance?
3. How would you handle one failed file?
4. How would you measure the bottleneck?
5. What would you do if the source system starts throttling?

19. Real Production Scenario: Pipeline Monitoring

PRODUCTION SCENARIO

Management wants email notifications whenever any production pipeline fails.

INTERVIEW QUESTION

How would you implement monitoring and alerting?

MY ANSWER

I would centralize operational monitoring rather than adding custom email logic to every pipeline. ADF pipeline and activity runs can be monitored, and Azure Monitor can be used to create alerting around relevant failure conditions. I would send notifications through the organization's approved alerting path, such as an action group connected to email or another incident channel. I would include useful context in the alert: factory, pipeline name, run ID, environment, failed activity, timestamp, and a link or reference for investigation.

I would also distinguish critical production failures from expected or retried transient failures to reduce alert noise. In parallel, I would maintain an audit table for business-level monitoring such as rows processed, duration, source file, batch ID, and validation status. The goal is not only to notify someone that a pipeline failed, but to give enough context to begin recovery quickly.

FOLLOW-UP QUESTIONS

1. Would you alert on every retry?
2. What information should an alert contain?
3. How would you avoid alert fatigue?
4. How would you monitor SLA breaches?
5. What would you keep in an audit table?

20. Real Production Scenario: Schema Drift

PRODUCTION SCENARIO

A source system adds new columns without informing the Data Engineering team.

INTERVIEW QUESTION

How would your pipeline handle schema changes without breaking?

MY ANSWER

I would first decide which schema changes are allowed and which require approval. For ingestion into a raw landing zone, I would generally preserve the source data and metadata so a new column does not destroy the batch. For ADF Mapping Data Flow, schema drift can be enabled so incoming fields not present in the projection can flow through the transformation; auto-mapping can be used at the sink when that behavior is appropriate.

However, I would not blindly accept every type change. I would compare the incoming schema with the expected contract and classify changes as additive, breaking, or incompatible. Additive columns might be captured automatically, while datatype changes or removed business-critical columns should trigger a controlled failure or review. I would log schema versions and alert the team when a change is detected. This gives flexibility without silently corrupting downstream tables.

FOLLOW-UP QUESTIONS

1. What is schema drift?
2. When would you allow a new column automatically?
3. Which changes should fail the pipeline?
4. How would you detect datatype changes?
5. How would you keep downstream reporting stable?

21. Real Production Scenario: File Archival

PRODUCTION SCENARIO

After successful processing, source files should be moved to an Archive folder. Failed files should go to a Reject folder.

INTERVIEW QUESTION

How would you automate this workflow?

MY ANSWER

I would make file lifecycle management part of the per-file processing pattern. After a successful validation and load, the pipeline would move or copy the source object into an Archive location and record the final status in an audit table. If validation or processing fails, the failure path would move the file into a Reject location, preserving enough metadata to investigate it. I would include the pipeline run ID, source file name, processing timestamp, and error message in the audit record.

Before moving a file, I would ensure the destination operation and target load are successful so that a file is never archived before its data is safely committed. I would also make the process replayable: an operator should be able to return a corrected rejected file to a controlled landing area and reprocess it without creating duplicates. Retention rules can then manage how long archived files remain available.

FOLLOW-UP QUESTIONS

1. When should a file be archived?
2. What should happen to a failed file?
3. How would you make reprocessing safe?
4. What metadata would you record?
5. How would you handle an archive operation failure?

22. Real Production Scenario: Watermark Table

PRODUCTION SCENARIO

Your source database doesn't have timestamps, but only increasing IDs.

INTERVIEW QUESTION

How would you perform incremental loading?

MY ANSWER

An increasing ID can be used as a watermark when it reliably represents newly created records. I would store the last successfully processed ID in a control table. At the beginning of each run, ADF would read that value and extract records where ID is greater than the stored watermark and less than or equal to a captured upper boundary. I prefer capturing an upper boundary at the beginning of the run because it gives the batch a stable range even if new records arrive while processing.

After the data is validated and committed, I would update the watermark to the highest successfully processed ID. I would also document an important limitation: an increasing ID generally detects inserts, not updates to older rows. If the business requires update capture, I would investigate CDC, change tracking, modified timestamps, or another source-side mechanism.

FOLLOW-UP QUESTIONS

1. Why capture an upper boundary?
2. Can an ID watermark detect updates?
3. Where would you store the last ID?
4. What happens if IDs have gaps?
5. How would you recover from a failed batch?

23. Real Production Scenario: Delta Load Failure

PRODUCTION SCENARIO

Yesterday's incremental pipeline failed, and today's execution must process both yesterday's and today's missing data.

INTERVIEW QUESTION

How would you design this solution?

MY ANSWER

I would avoid using the calendar date itself as proof that a batch was processed. Instead, I would maintain a durable checkpoint representing the last successful source boundary. If yesterday's run failed before the checkpoint advanced, today's run would read the same starting watermark and process the complete uncommitted window. If the source supports an upper boundary, I would capture it for the current run and process the range deterministically.

The sink must be idempotent so replaying yesterday's records does not create duplicates. I would record each batch in a control table with **STARTED**, **SUCCESS**, or **FAILED** status and only update the watermark after the target load and validations complete. This approach naturally supports recovery: the next run sees the last successful checkpoint rather than assuming the previous calendar period completed.

FOLLOW-UP QUESTIONS

1. Why should the watermark not advance after failure?
2. How would you prevent duplicate records during replay?
3. What would you store in the batch-control table?
4. How would you process a week of missed data?
5. How would you distinguish a retry from a new business batch?

24. Real Production Scenario: Millions of Small Files

PRODUCTION SCENARIO

Thousands of small files are generated every day, causing poor pipeline performance.

INTERVIEW QUESTION

How would you optimize the ingestion process?

MY ANSWER

I would first confirm that small-file overhead is actually the bottleneck. Thousands of tiny files increase listing, connection, metadata, and transaction overhead and can also hurt downstream distributed processing. If the upstream system can be changed, I would prefer fewer larger files with a sensible target size and a stable naming/partitioning strategy. If the files must arrive individually, I would use controlled parallel ingestion and then compact them into larger analytical files such as Parquet in the processing layer.

I would avoid creating excessive partitions based on very high-cardinality columns. For recovery, each source file should still be tracked by name and checksum or another identifier so compaction does not remove traceability. I would measure file-count reduction, ingestion duration, downstream scan time, and cost to confirm the change improved the overall platform rather than just one activity.

FOLLOW-UP QUESTIONS

1. Why are small files expensive?
2. Would you combine files in ADF or Databricks?
3. How would you preserve traceability after compaction?
4. How would you choose a partition strategy?
5. How would you prove the optimization worked?

25. Real Production Scenario: Data Quality

PRODUCTION SCENARIO

Customer IDs should never be null, and duplicate Order IDs must be rejected before loading.

INTERVIEW QUESTION

How would you implement these quality checks?

MY ANSWER

I would create explicit validation rules before the final target load. For CustomerID, I would check both NULL values and, where relevant, empty or invalid values. For OrderID, I would group by the business key and identify counts greater than one. Invalid records would be routed to a quarantine or reject dataset with a reason code, while valid records would continue to the target. I would store the number of input rows, valid rows, rejected rows, and duplicate keys in an audit table.

For critical rules, the pipeline should fail the business load if the quality threshold is violated rather than publishing incomplete data as if it were correct. I would also distinguish between a technical failure and a data-quality failure so support teams know whether to restart the pipeline or contact the source owner. The rules should be metadata-driven when many datasets need similar validation.

FOLLOW-UP QUESTIONS

1. Would you reject the entire batch or only bad rows?
2. How would you identify duplicate OrderIDs?
3. Where would you store rejected records?
4. How would you report data-quality trends?
5. How would you make these checks reusable?

26. Real Production Scenario: Secure S3 Access

PRODUCTION SCENARIO

Azure Data Factory needs to read files from a private AWS S3 bucket.

INTERVIEW QUESTION

What is the most secure way to establish connectivity?

MY ANSWER

I would start with the security requirements and the authentication method supported by the exact ADF Amazon S3 connector version being used. The key principle is least privilege: ADF should have only the permissions required to list and read the required bucket or prefix. I would keep credentials out of pipeline code and use Azure Key Vault to protect secrets, with ADF managed identity used to retrieve them.

I would also restrict the AWS policy to the required objects rather than granting broad S3 access, enable appropriate logging on the AWS side, and review network requirements for the organization's cross-cloud architecture. Before production, I would test access with a non-production prefix and verify that write or delete permissions are not accidentally granted. The connector documentation should be checked for the currently supported S3 authentication options before implementation.

FOLLOW-UP QUESTIONS

1. What S3 permissions does the pipeline actually need?
2. Where would you store any required secret?
3. How would you prevent accidental write access?
4. What would you monitor in AWS?
5. How would you test the connection safely?

27. Real Production Scenario: Slowly Running SQL Query

PRODUCTION SCENARIO

Your Copy Activity is taking much longer than expected because the source SQL query is slow.

INTERVIEW QUESTION

How would you troubleshoot and optimize it?

MY ANSWER

I would separate source-query time from ADF transfer time. First I would run the exact query directly in the source database and inspect the execution plan, indexes, filters, joins, scans, blocking, and statistics. I would make sure the incremental predicate is sargable and uses an indexed column where possible. I would avoid selecting unused columns and reduce unnecessary joins or transformations in the extraction query.

If the source is under heavy load, I would also check whether the query is competing with operational workloads. On the ADF side, I would compare source read throughput with sink write throughput and verify Integration Runtime capacity. For large extracts, I would investigate partitioned or parallel reads when supported. I would benchmark each change separately so I can demonstrate whether the improvement came from SQL tuning, source-side capacity, network throughput, or sink configuration.

FOLLOW-UP QUESTIONS

1. How would you inspect the SQL execution plan?
2. Why is an indexed watermark useful?
3. How would you separate source and ADF bottlenecks?
4. When would you use partitioned extraction?
5. How would you validate the optimization?

28. Real Production Scenario: Daily Full Load vs Incremental Load

PRODUCTION SCENARIO

Business currently performs full data loads every day, but the data volume has increased significantly.

INTERVIEW QUESTION

Would you continue using full loads or switch to incremental loads? Why?

MY ANSWER

I would evaluate the workload rather than automatically replacing every full load. If only a small percentage of records changes each day, incremental loading will usually reduce source reads, network movement, compute, and processing time. I would first identify whether the source has a reliable watermark, CDC, change tracking, or another change-detection mechanism. I would then design an incremental path with a safe checkpoint, idempotent target logic, and recovery handling.

I would still consider periodic reconciliation or full refreshes when business requirements justify them, because incremental pipelines can miss changes if the source change mechanism is incomplete. The decision should be based on data volume, change rate, source capabilities, SLA, recovery requirements, and operational complexity. I would prove the design with a representative benchmark before changing production.

FOLLOW-UP QUESTIONS

1. What makes a good incremental source?
2. When is a full load still useful?
3. How would you reconcile an incremental target?
4. How would you handle deletes?
5. What risks come with incremental processing?

29. Real Production Scenario: Metadata-Driven ETL

PRODUCTION SCENARIO

New source tables are added every month, and developers don't want to modify pipelines every time.

INTERVIEW QUESTION

How would you design a metadata-driven ETL framework?

MY ANSWER

I would create a configuration layer that describes each source and target rather than embedding the details in pipeline activities. A typical metadata record would contain source connection or system, schema, table, target path/table, load type, watermark column, primary key, partition strategy, active flag, and validation rules. A master pipeline would read the active records and call a reusable child pipeline for each dataset.

The child pipeline would receive parameters and dynamically build the source query, target path, and audit information. I would also maintain execution metadata separately so configuration does not become mixed with operational history. Adding a new table would then be a controlled metadata change followed by validation, rather than creating another pipeline. I would protect the metadata tables, version their changes, and test new configurations in lower environments before promoting them to production.

FOLLOW-UP QUESTIONS

1. What fields belong in the metadata table?
2. How would the child pipeline receive parameters?
3. How would you handle different load types?
4. How would you validate a new metadata record?
5. How would you prevent a bad configuration from affecting all tables?

30. Real Production Scenario: End-to-End Failure Analysis

PRODUCTION SCENARIO

Business reports that today's dashboard shows incorrect sales numbers.

INTERVIEW QUESTION

How would you investigate the issue from source system to final reporting layer?

MY ANSWER

I would investigate from the dashboard backward to the source, using the batch and data timestamps to identify the affected processing window. First I would confirm exactly what metric is wrong and compare the dashboard result with the semantic or warehouse query. Then I would check whether the curated table contains the expected rows and totals. If not, I would move upstream to the transformation layer, checking joins, filters, deduplication, late-arriving data, and failed batches.

Next I would validate the raw landing layer against the source counts or control totals. I would also inspect ADF run history, Databricks job logs, SQL load status, and data-quality results for the same batch ID. If the data is correct in the warehouse but wrong in the dashboard, I would investigate semantic-model filters, relationships, refresh status, or aggregation logic. I would document the root cause, affected time window, corrective action, and validation evidence before closing the incident.

FOLLOW-UP QUESTIONS

1. How would you identify the first layer where the numbers diverge?
2. What reconciliation checks would you run?
3. How would batch IDs help?
4. What if the warehouse is correct but the dashboard is wrong?
5. What evidence would you document after fixing the issue?